Relatório: Configuração Centralizada com Spring Cloud

1. Conexão dos Serviços ao Servidor

Nesta arquitetura, o Config Server atua como a fonte central de propriedades para os microsserviços. A conexão ocorre por meio da dependência `Spring Cloud Config Client`, incluída em cada aplicação cliente (cliente-vendas, cliente-estoque, cliente-relatorios).

Durante a inicialização, antes de levantar o contexto principal do Spring, o cliente consulta a propriedade `spring.config.import` definida no seu arquivo local. Essa instrução aponta para o endereço do servidor (neste projeto, a porta 8888). O cliente realiza uma requisição HTTP REST ao Config Server para recuperar as suas propriedades de configuração específicas.

2. Função das Propriedades (application.properties)

As configurações foram divididas entre o servidor e os clientes para garantir o funcionamento da arquitetura.

No Config Server:

- `server.port=8888`: Define a porta de execução padrão do servidor de configuração.
- `spring.cloud.config.server.git.uri`: Informa o endereço do repositório Git remoto (GitHub) onde os arquivos centralizados estão armazenados.
- `spring.cloud.config.server.git.clone-on-start=true`: Força o servidor a clonar o repositório no momento da sua inicialização, otimizando o tempo de resposta na primeira requisição de um cliente.

Nos Clientes:

- `spring.application.name`: Identifica a aplicação (ex: `cliente-vendas`). O servidor utiliza esse nome para localizar o arquivo correspondente no repositório.
- `spring.profiles.active=dev`: Define o ambiente de execução. Combinado com o nome da aplicação, faz o servidor buscar o arquivo `cliente-vendas-dev.properties`.
- `spring.config.import`: Informa a URI do Config Server (ex: `optional:configserver:http://localhost:8888`) para o cliente buscar os dados.

3. Fluxo de Inicialização e Obtenção das Configurações

A ordem de inicialização é estrita: o Config Server deve ser o primeiro a ser executado.

1. O servidor inicia, conecta-se ao GitHub e clona o repositório contendo as configurações.
2. Os serviços dos clientes são iniciados.
3. Na fase inicial de *bootstrap*, o cliente requisita ao servidor as configurações referentes ao seu nome e perfil ativo.
4. O servidor localiza o arquivo correspondente (ex: `cliente-estoque-dev.properties`), converte as propriedades para um formato JSON e envia a resposta.
5. O cliente injeta esses valores recebidos nas variáveis da aplicação utilizando a anotação `@Value`.

Anotações Utilizadas:

- `@EnableConfigServer` (no servidor): Habilita o comportamento de um servidor central de configuração na aplicação Spring Boot.
- `@Value` (no cliente): Injeta os valores recuperados do servidor diretamente nas variáveis da classe.
- `@RefreshScope` (no cliente): Permite a atualização dinâmica das variáveis configuradas via `@Value` em tempo de execução, recriando os componentes após a chamada do endpoint `/refresh`.

4. Testes e Atualização Dinâmica

Os testes validaram a obtenção das configurações e a atualização em tempo real utilizando o Spring Boot Actuator.

Acesso Inicial:

- Foi verificado o status do servidor via `http://localhost:8888/actuator/health` (retornando "UP").
- O acesso direto ao endpoint do cliente (ex: `http://localhost:8081/mensagem`) retornou o texto definido no GitHub.

Atualização (Refresh):

1. O valor da propriedade foi alterado no repositório Git.
2. Uma requisição POST com corpo vazio foi enviada para o endpoint `http://localhost:8081/actuator/refresh` via Postman. Requisições GET retornaram erro 405 (Method Not Allowed), confirmando que a operação exige POST.
3. O endpoint retornou status 200 OK e a lista de propriedades atualizadas.
4. A página foi atualizada no navegador, e o cliente exibiu o novo valor sem a necessidade de reiniciar a aplicação (zero downtime).